Sentiment Analysis on IMDb Movie Reviews

A Comparative Study of Classical and Transformer-Based Approaches

MAHIDDINE Islem Amine
TALAHARI Yassine
YOUSFI Nacer
ZORGANI Alaaeddine

Machine Learning - University of Boumerdes - Department of Computer Science - December 2025

1. Introduction

Sentiment analysis has become one of the most widely explored tasks in Natural Language Processing, largely because it sits at the intersection of linguistic nuance and practical machine learning applications. In this project, we focus on **the IMDb 50K movie review dataset**,, aiming to automatically classify reviews as positive or negative. To do so, we systematically explore a range of approaches from classical machine learning algorithms powered by TF-IDF features, to more expressive neural models, all the way to modern Transformer-based architectures. Our pipeline begins with extensive text preprocessing and vectorization before training and evaluating several models: Naive Bayes, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and an Artificial Neural Network (ANN). We then contrast these traditional techniques with a fine-tuned RoBERTa model, which represents the current state of the art in text understanding. By comparing performance across all methods, the study highlights both the strengths and limitations of classical TF-IDF models and illustrates why exploiting the deep contextual representations learned by pretrained Transformer models has become the dominant strategy for contemporary NLP tasks like sentiment analysis.

2. Data + Training Pipeline

This project focuses on comparing the performance of several machine learning models, along with an additional deep learning approach, to evaluate how well each one handles the classical natural language processing task of sentiment analysis. We use the IMDb 50K Movie Reviews dataset from Kaggle, which provides a balanced and diverse collection of user-written reviews labeled as positive or negative. Our goal is to build a consistent pipeline that begins with thorough text preprocessing and proceeds through vectorization, model training, and evaluation. By applying the same pipeline to different algorithms, we can fairly assess their respective strengths, weaknesses, and suitability for sentiment classification. This setup also allows us to highlight the contrast between traditional TF-IDF-based methods and modern Transformer-based models, giving a clear picture of how model choice influences performance on real-world text data.

The pipeline we came up with is the following:

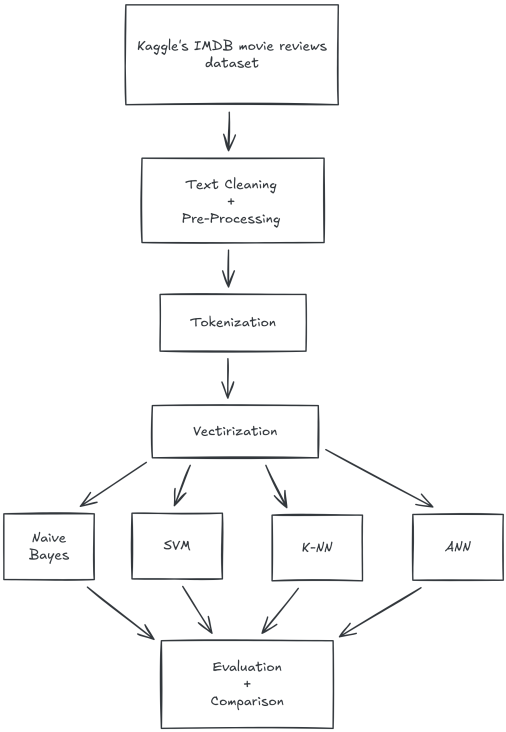

Figure 1: A chart illustrating the data preprocessing + training pipeline.

2.1. Data Cleaning:

We began by cleaning the raw text using Python's re library to remove HTML tags, punctuation, numbers, and special characters. The text was then converted to lowercase for consistency. Additionally, stopwords were filtered out using the nltk.corpus.stopwords module, and lemmatization was applied with WordNetLemmatizer to reduce words to their base form.

2.2. Tokenization, Stopword Removal and Lemmatization:

Each review was tokenized into individual words. Common words that do not carry significant meaning, known as *stopwords* (e.g., "the", "is", "in"), were removed using the NLTK library. This step helps reduce noise and focus the analysis on meaningful terms. Yassine applied lemmatization to reduce words to their base or dictionary form. For example, "running" becomes "run" and

“better” becomes “good”. This ensures that different grammatical forms of a word are treated as the same token.

2.3. Vectorization

Now that we have clean data, we vectorize it through generating the TF-IDF representation which we’ll get into in more details later on but this serves as the input we feed to our models through the training process.

2.4. Training Models on the data

We now take our vectorized representation of the dataset and feed it into different machine learning models (mostly classic ones). These include Naive Bayes, a Support Vector Machine model, a K-Nearest Neighbors Model and an ANN. We ensure the hyperparameters for each training process are optimal using appropriate techniques so that we ensure that we will be comparing the most optimal models of each category later on.

2.5. Evaluation and Comparison

The evaluation steps here are mostly based on the confustion matrix, they require calculating the accuracy, recall, precision and F1 score for each model and see how they stack up against each other so that a judgment concerning the goal of this project which aims at determining which model is the best one can be made.

3. TF-IDF

3.1. Definition and Theory

After cleaning and normalizing the text, we converted it into numerical representations using the **TF-IDF (Term Frequency–Inverse Document Frequency)** method. TF-IDF helps measure the importance of words in each review relative to the entire dataset, making it more suitable for text classification.

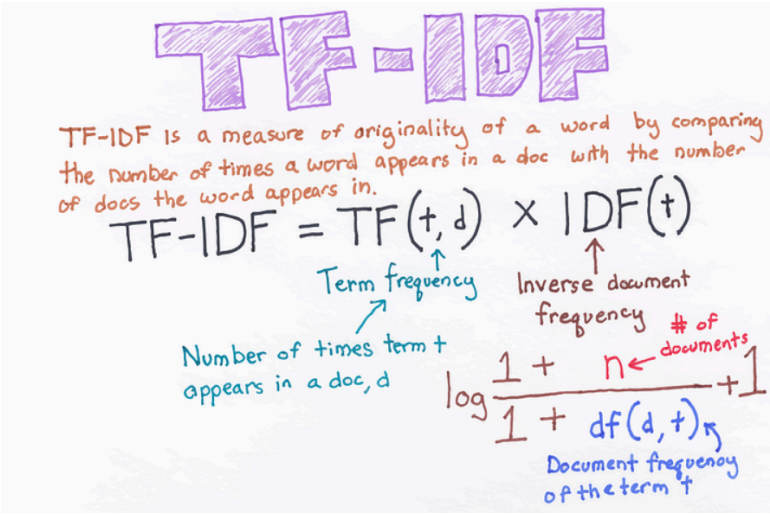

Figure 2: A diagram illustrating the theory and formula of TF-IDF

3.2. Tradeoffs

3.2.1. Advantages

- Captures term importance by downweighting common words that appear across many documents.
- Simple and computationally efficient for transforming text into numerical features.
- Effective for traditional ML models in text classification tasks like sentiment analysis.

3.2.2. Disadvantages

- Ignores word order, semantics, and context, treating text as a "bag of words."
- Can result in high-dimensional sparse vectors, leading to the curse of dimensionality.
- Sensitive to document length and may not handle synonyms or polysemy well.

4. Naive Bayes

4.1. Definition and Theory

These preprocessing and vectorization steps prepare the data for the next stage: training the Naive Bayes classifier. The theorem shown below helps calculate the probability of a message belonging to a certain category. After preprocessing and vectorizing the text data with TF-IDF, we trained a **Naive Bayes classifier** using the **MultinomialNB** model from the **scikit-learn** library. This algorithm is well-suited for text classification tasks, as it assumes that features (words) are conditionally independent given the class label.

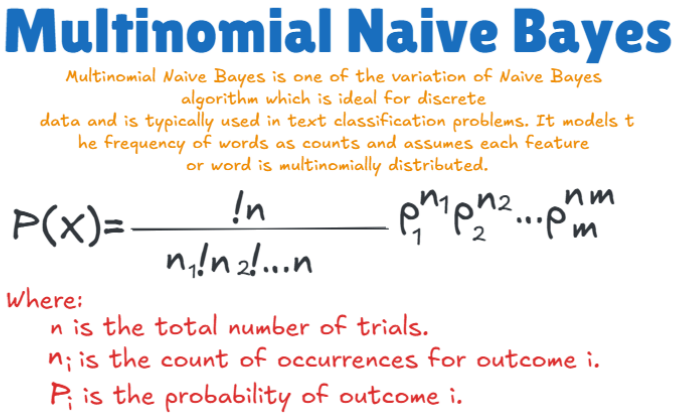

Figure 3: A diagram illustrating the theory and formula of Naive Bayes

The dataset was split into training and testing sets using **train_test_split**. The model was then trained on the TF-IDF vectors and corresponding sentiment labels. Once trained, predictions were made on the test set, and performance was evaluated using metrics from **sklearn.metrics**, including **accuracy_score**, **confusion_matrix**, and **classification_report**.

The model achieved an accuracy of approximately **85.5%**, demonstrating strong performance for a simple yet efficient probabilistic approach.

4.2. Tradeoffs

4.2.1. Advantages

- Extremely fast and efficient for training and prediction, especially on large datasets.
- Performs well on text data despite the naive independence assumption.
- Handles high-dimensional features like TF-IDF vectors effectively.

4.2.2. Disadvantages

- The independence assumption is often violated in real text, leading to suboptimal modeling of dependencies.
- Suffers from zero-probability issues for unseen words, requiring smoothing techniques.
- Less effective for capturing complex patterns compared to more advanced models.

4.3. Naive Bayes in the context of our project

In our implementation, we used the Multinomial Naive Bayes classifier from scikit-learn, trained on TF-IDF vectors limited to 5,000 features. The model was fit on the training split after preprocessing, with predictions evaluated on the test set. This probabilistic approach provided a quick baseline, leveraging word frequencies to classify sentiments.

4.4. Results

The Naive Bayes model achieved an accuracy of 86.74% on the test set, with balanced precision and recall across classes. The confusion matrix showed minimal bias, though errors often stemmed from overlapping word usages in sarcastic reviews.

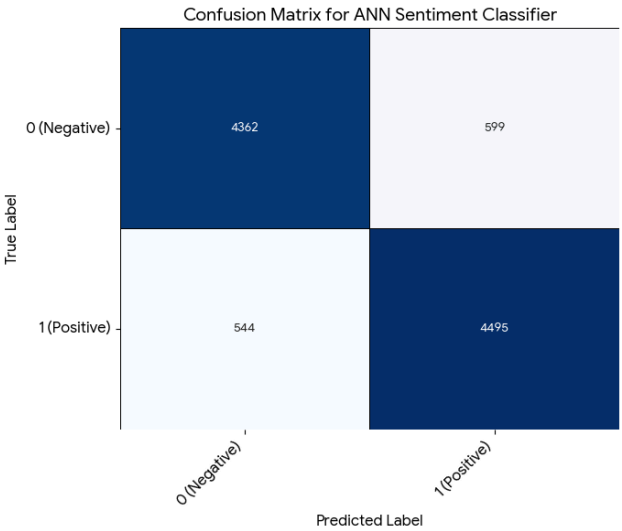

Figure 4: Naive Bayes Confusion Matrix

5. Support Vector Machines (SVM)

5.1. Definition

Support Vector Machines (SVM) are supervised learning algorithms designed to perform classification by identifying the optimal decision boundary called a **hyperplane** that separates data points from different classes. The optimal hyperplane is the one that maximizes the **margin**, i.e., the distance between the boundary and the closest samples of each class, known as **support vectors**. By relying only on these critical points, SVMs achieve strong generalization and robustness, especially in high-dimensional feature spaces.

5.2. Theory

The general functioning of an SVM can be described step by step:

1. **Feature Vectorization**: Input samples are converted into numerical feature vectors. For text classification we used **TF-IDF** (Term Frequency–Inverse Document Frequency) to transform raw reviews into fixed-length sparse vectors. In our pipeline the TF-IDF vectorizer was limited to the **5,000** most frequent terms to reduce dimensionality and noise.
2. **Finding the Hyperplane**: The algorithm attempts to find a hyperplane that separates classes with the maximum margin. For linearly separable

data this hyperplane is represented by parameters $\boldsymbol{w}$ (weights) and b (bias) forming the decision function:

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b.$$

The margin is defined as the distance between the hyperplane and the nearest data points of any class; these closest points are the **support vectors**.

3. **The Kernel Trick**: If data is not linearly separable in the original space, SVMs apply a **kernel function** to implicitly map data into a higher-dimensional feature space where a linear hyperplane may separate classes. This avoids explicitly computing the high-dimensional mapping.
4. **Soft Margin**: Real-world data often contains noise and overlap between classes. SVMs allow some misclassifications using a **soft margin** controlled by the regularization parameter C, which balances margin width against classification error.

Mathematically, training a binary SVM is formulated as a convex optimization problem that maximizes the margin while penalizing misclassifications. When kernels are introduced, the optimization operates in the dual space and relies on kernel evaluations $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ to compute inner products in the transformed feature space without explicit mapping.

Common kernel functions include:

- **Linear Kernel**: Effective for high-dimensional sparse data (e.g., TF-IDF).
- **RBF (Radial Basis Function) Kernel**: Captures complex non-linear relationships.
- **Polynomial Kernel**: Models polynomial decision boundaries of adjustable degree.

Kernel Type	Best For	Key Characteristics
Linear	High-dimensional data (like text)	Fast, less prone to overfitting; suitable for sparse TF-IDF representations.
RBF	Complex, non-linear relationships	Flexible, can model complex boundaries; requires tuning of gamma.

Kernel Type	Best For	Key Characteristics
Polynomial	Non-linear data	Can model various curved shapes; degree controls flexibility.

5.3. Tradeoffs

5.3.1. Advantages

- Effective in high-dimensional spaces; SVMs perform well on sparse TF-IDF vectors and text features.
- Robust and stable. The margin maximization principle reduces the influence of noisy or non-informative points.
- Memory efficient. Only support vectors need to be stored after training.
- Excellent performance. Often matches or outperforms more complex models on classical text classification tasks.

5.3.2. Disadvantages

- Training time can be slow and resource-intensive on very large datasets.
- Parameter tuning is critical. Performance depends on hyperparameters such as C, kernel choice, and kernel-specific parameters (e.g., gamma).
- Interpretability is reduced with non-linear kernels, making it harder to explain decisions.

5.4. SVM in the context of our project

In our project we implemented a **Linear SVM** classifier using Scikit-learn's `LinearSVC`. The processing pipeline was:

1. Text cleaning and normalization (lowercasing, punctuation removal, basic tokenization).
2. Feature extraction via **TF-IDF** with a vocabulary restricted to the top **5,000** most frequent terms.
3. Training a `LinearSVC` on the TF-IDF vectors of the training set.
4. Evaluating model performance on a held-out test set and inspecting coefficients to determine discriminative features.

By inspecting the learned weight coefficients, we identified the most discriminative tokens: positive-indicating tokens such as *"favorite"*, *"brilliant"*, and *"excellent"*, and negative-indicating tokens such as *"worst"*, *"awful"*, and *"boring"*.

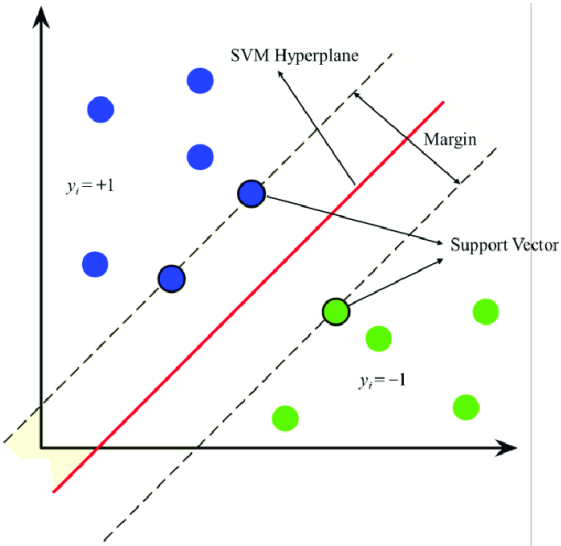

Figure 5: A diagram illustrating how SVM works

5.5. Results

The implemented Linear SVM achieved an **accuracy of 88.18%** on the test set. The confusion matrix (shown below) confirms a balanced performance across classes with near-symmetric false positive and false negative rates.

Analysis of errors shows that remaining misclassifications primarily originate from subtle linguistic phenomena:

- **Sarcasm and irony**, where positive wording masks negative sentiment.
- **Complex negation** (e.g., double negatives) or long-distance dependencies.
- **Ambiguous or contextual expressions** that require deeper semantic understanding.

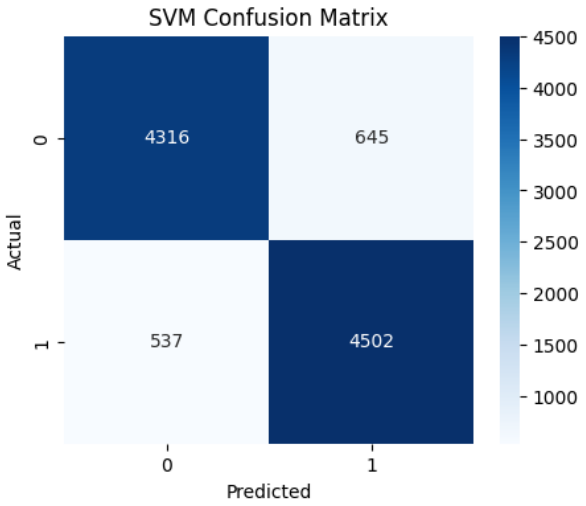

Figure 6: The confusion Matrix for Amine's SVM model.

6. K-Nearest Neighbors (K-NN)

6.1. Definition

The K-Nearest Neighbors (KNN) algorithm is a classic example of an instance-based supervised learning method primarily utilized for classification tasks. It operates on a direct, non-parametric logic, avoiding the construction of a complex explicit model during the training phase. Instead, the algorithm relies entirely on the set of already known examples to make decisions, founded on the intuitive hypothesis that data points that are sufficiently similar generally belong to the same category.

6.2. Theory

The KNN process is conceptually defined by four key stages. First, the algorithm requires the numerical representation of the data, where each film review is converted into a feature vector, in this project's case, using the TF-IDF method. When predicting the class for a new, unlabeled input x, the model calculates the distance between x and all existing critiques y. This measurement formalizes the similarity assumption, where the distance $d(x, y)$ is typically calculated using the Euclidean metric:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Following the distance calculation, the process selects the K critiques that are the nearest neighbors. The final prediction is then determined by a majority vote, where the most frequent class among the K selected neighbors is assigned to the new critique. This entire workflow mimics a form of human inductive reasoning.

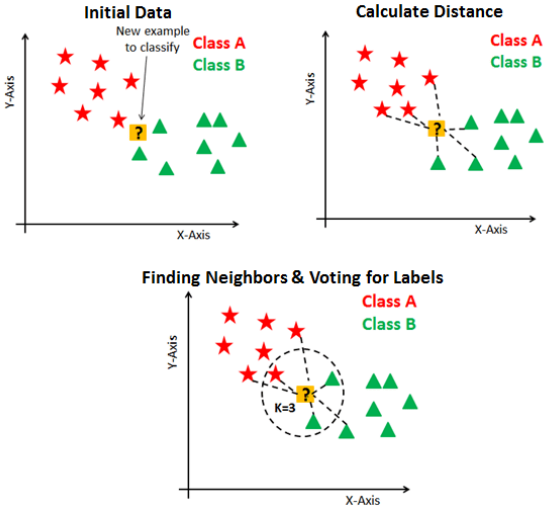

Figure 7: How K-NN works

6.3. Tradeoffs

6.3.1. Advantages

- The algorithm is exceptionally simple to understand and explain, requiring no complex underlying mathematical model.
- It involves no costly training phase; the main computational work is deferred until the moment of prediction.
- KNN offers high polyvalence, being readily applicable to both classification and regression tasks.

6.3.2. Disadvantages

- It becomes slow and computationally expensive on large datasets, as every new prediction necessitates comparing the input to potentially thousands of existing examples.
- Performance is highly sensitive to the choice of the parameter K; a suboptimal value can result in a significant loss of precision.
- The model is ineffective in high-dimensional feature spaces, which is a frequent limitation when working with text data vectorized by methods like TF-IDF.

6.4. K-NN in the context of our project

The project implemented KNN as the initial machine learning step following the preparation of the IMDb dataset. The data pipeline first focused on data cleaning, including the systematic removal of HTML tags, filtering of non-alphabetic characters, standardizing all text to lowercase, suppressing common stopwords, and applying stemming to reduce words to their common root. The cleaned text was then converted via TF-IDF vectorization, which translated each review into a vector reflecting the unique importance of each word relative to the entire corpus. The final model training involved dividing the dataset into an 80% training set and a 20% test set, followed by an optimization stage where several values of K were tested to identify the optimal hyperparameter. This process ensured the model was built on the robust principles of vector similarity for classification.

6.5. Results

The K-Nearest Neighbors model achieved its objective by coherently distinguishing between positive and negative reviews based on the vectorized text input. Although this simple algorithmic choice is generally recognized as not being the most high-performing for voluminous text data, the model successfully provided an operational foundation for understanding the core logic of text classification. The analysis confirms that the KNN method can serve as a robust, albeit basic, starting point for automated natural language processing.

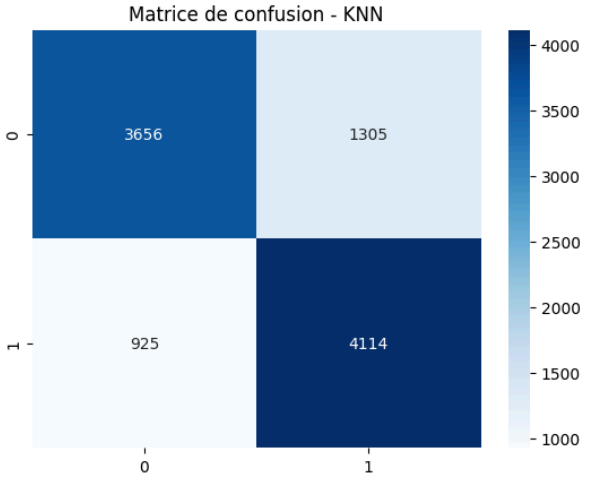

Figure 8: K-NN Confusion Matrix

7. Artificial Neural Networks (ANN)

7.1. Definition

Artificial Neural Networks are computational models composed of layers of interconnected processing units called neurons, which collectively approximate complex nonlinear functions. Each neuron applies a weighted linear transformation to its inputs followed by a nonlinear activation function, allowing the network to represent highly non-linear de- pendencies between variables. Training is performed via optimization techniques such as stochastic gradient descent, guided by the backpropagation algorithm, which efficiently computes gradients of a loss function with respect to all network parameters. ANNs are universal function approximators, meaning they can approximate any measurable func- tion given sufficient depth, width, and appropriate training; however, they require large data and careful regularization to generalize effectively.

7.2. Theory

From a theoretical perspective, an ANN can be viewed as a parametric function $f : \mathbb{R}^n \to \mathbb{R}^m$ composed of successive transformations $f(x) = f^{(L)}\big(f^{(L-1)}\big(\ldots f^{(1)}(x)\big)\big)$, where each layer $f^{(l)}(x) = \sigma\big(W^{(l)}x + b^{(l)}\big)$ applies

an affine map followed by a nonlinear activation function σ. The presence of nonlinearity is essential: if σ were linear, the entire network would collapse into a single affine transformation. Nonlinear activations such as $\text{relu}(z) = \max(0, z)$ or $\sigma(z) = \frac{1}{1+e^{-z}}$ allow the model to construct piecewise-linear or smoothly curved decision boundaries. The universal approximation theorem formalizes this, showing that a network with a sufficiently large hidden layer and any non-constant, bounded, and continuous activation σ can approximate any continuous function on a compact domain. This places neural networks within the broader theory of nonlinear function approximation, where depth, activation smoothness, and parameterization govern their expressive capacity.

7.3. Tradeoffs

7.3.1. Advantages

- Artificial Neural Networks possess the inherent capacity to model highly complex, non-linear relationships within unstructured data, allowing them to solve intricate problems like image recognition that are often impossible for traditional linear algorithms.
- Due to their parallel distributed processing architecture, these networks exhibit significant fault tolerance and can maintain overall performance even if the input data contains substantial noise or if individual neurons fail.
- Once properly trained, neural networks demonstrate a strong ability to generalize patterns from training sets to infer accurate results from entirely new data, making them highly effective for predictive modeling in dynamic environments.

7.3.2. Disadvantages

- The internal mechanics of deep neural networks are often mathematically opaque, creating a “black box” problem that makes it incredibly difficult for humans to interpret exactly how specific inputs led to a specific decision.
- Achieving state-of-art performance typically requires massive amounts of labeled training data and expensive computational hardware, creating a high barrier to entry compared to simpler, more efficient statistical models.
- Without careful architectural tuning, ANNs have a strong tendency to overfit by memorizing the statistical noise of the training data rather than learning underlying concepts, which results in poor performance when applied to real-world scenarios.

7.4. ANNs in the context of our project

The true power of the neural network emerges within its hidden layers, where non-linear activation functions like ReLU are applied to progressively transform the high-dimensional input vector, whether it is a sparse TF-IDF array or dense word embeddings into a concise, abstract semantic representation. Through the continuous adjustment of thousands of internal weights, the network learns to identify intricate composite features and distinguish complex semantic patterns, such as sarcasm or negation, which are then channeled to the final output neuron to generate the single probability score for binary classification.

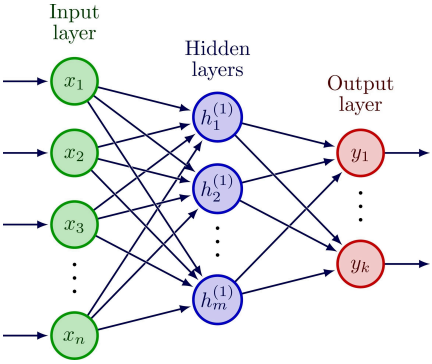

Figure 9: A diagram illustrating the layered structure of the ANN.

7.5. Results

The Artificial Neural Network achieved a highly successful initial result on the binary sentiment classification task, securing an overall accuracy of 88.6%. This strong outcome is reinforced by the balanced performance demonstrated across the confusion matrix and classification report, where the near-identical F1-scores of 0.887 and 0.884 for the positive and negative classes, respectively, confirm the model exhibits no significant bias toward either sentiment. While this level of robustness is excellent for a first-pass model, the remaining 11.4% error margin highlights the complex, nuanced textual examples that the current architecture, relying on fixed-length vectorized inputs, struggles to resolve. Moving forward, the most impactful next step involves exploring Transformer-based pretrained models which leverage their own context-rich embedding spaces via mechanisms like self-attention to significantly enhance semantic understanding and capture complex linguistic dependencies. The optimal strategy would be to finetune a powerful model such as BERT

directly on the IMDb corpus, thereby leveraging its immense general language knowledge to tackle the specific sentiment task. Alternatively, substantial gains could be realized by conducting a more robust and extensive pretreatment of the current dataset, or by seeking a better-labeled dataset altogether, ensuring that the input supplied to any subsequent model is maximally clean and representative of true linguistic variation.

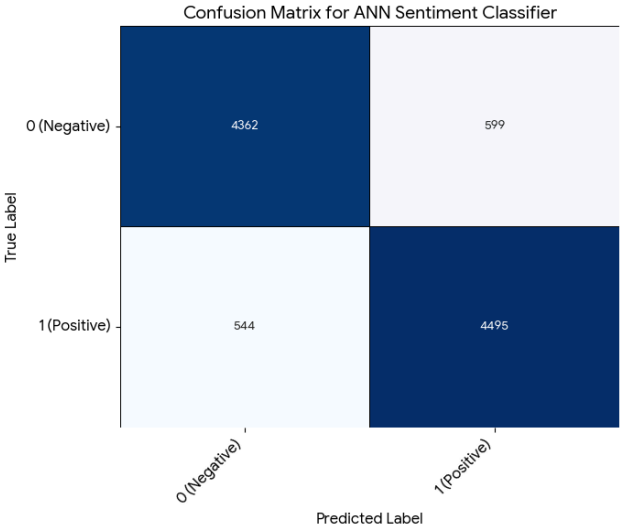

Figure 10: ANN Confusion Matrix

8. Deep Learning: Fine-Tuning Pre-Treained Transformer Based Models

8.1. Background: The Transformer Architecture

Transformers represent a major shift in how modern NLP models understand language, replacing the sequential, step by step processing of RNNs with a fully attention-based architecture capable of modeling global context in a single pass. At their core lies the self-attention mechanism, which allows every token in a sentence to directly attend to every other token, assigning learned importance weights that reveal which words are relevant to interpreting the current one. This eliminates the bottleneck of recurrence, enabling transformers to capture long-range dependencies such as negation, sarcasm, or multi-clause reasoning with far greater reliability than LSTMs or

GRUs. Positional encodings reintroduce order information, while stacked layers of multi-head attention and feed-forward networks give the architecture the depth and flexibility needed to learn highly abstract linguistic patterns. Because transformers can be pretrained on massive corpora via self-supervised objectives like masked language modeling, they acquire broad general-purpose language understanding before being applied to any specific task. As a result, it has become highly effective to exploit this architecture by fine-tuning pre-trained transformer-based models, tailoring their learned representations to tasks such as sentiment analysis for substantial performance gains.

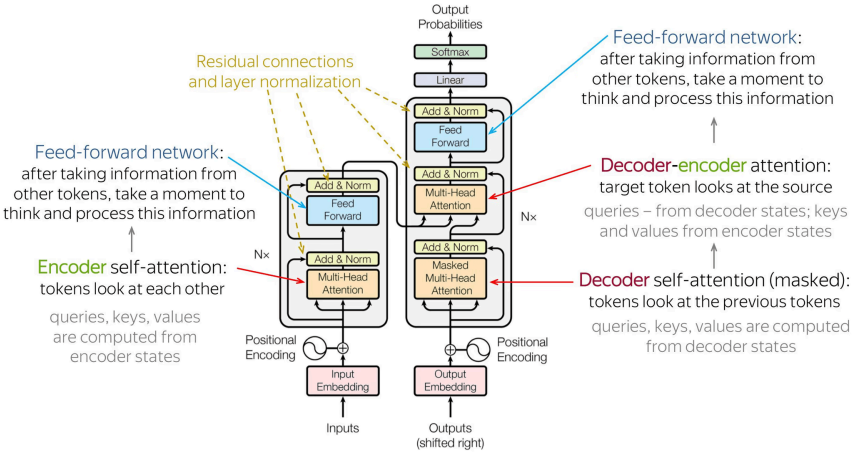

Figure 11: The Transformer Architecture

8.2. Definition (Fine-Tuning)

Fine-tuning is a highly efficient transfer learning paradigm where a massive neural network, pre-trained for weeks on a general language corpus, is adapted to a specific downstream task like sentiment analysis by adjusting its weights for a short period. This method is exceptionally practical and popular because it dramatically circumvents the resource-intensive process of training a comparable deep learning model from scratch, which would otherwise require enormous datasets and prohibitively expensive hardware clusters for initialization. By using a pre-existing foundation of learned linguistic knowledge, fine-tuning effectively transfers deep comprehension of language structure and grammar, allowing the model to quickly optimize for the subtle semantic demands of the IMDb movie reviews.

8.3. Theory

The theoretical basis of fine-tuning relies on the assumption that the extensive knowledge acquired during pre-training specifically related to syntax, context, and vocabulary can be reused. The process begins with a model initialized with the pretrained weights θ_{pre} and often involves attaching a new, randomly initialized classification head for the specific task

. During fine-tuning, the optimization process is driven by the task-specific loss function L_{task}, and the weights are updated using a smaller learning rate η, as represented by the general update rule: $\theta_{\text{new}} = \theta_{\text{pre}} - \eta \nabla L_{\text{task}(\theta_{\text{pre}})}$. Before processing, the raw text enters the Tokenizer, which uses algorithms like Byte-Pair Encoding (BPE) to convert words and sub-words into discrete numerical identifiers (input IDs), while also generating the attention mask to handle variable-length inputs via padding/truncation. The subsequent finetuning stage utilizes high-level abstractions, such as the Hugging Face Trainer object, which handles all theoretical baggage, including batch management, gradient accumulation, learning rate scheduling (a crucial training argument), and efficient gradient descent across the entire network architecture.

8.4. Tradeoffs

8.4.1. Advantages

- Knowledge Transfer: The model leverages deep, generalized linguistic knowledge learned from vast corpora, resulting in dramatically superior performance on complex NLU tasks compared to models trained solely on limited task-specific data.
- Resource Efficiency: Fine-tuning significantly reduces computational time and hardware cost, requiring hours on a single dedicated GPU instead of weeks on multiple specialized hardware units necessary for foundational pre-training.
- Data Efficiency: High performance can be achieved with relatively small amounts of labeled training data for the specific task, making it highly practical for scenarios where large, annotated datasets are unavailable.

8.4.2. Disadvantages

- Resource Consumption: The size of the pretrained models (often billions of parameters) demands considerable GPU memory (VRAM) simply for loading the model, imposing a non-trivial baseline hardware requirement.

- Hyperparameter Sensitivity: Performance is highly sensitive to the chosen learning rate and batch size during finetuning, often requiring careful search to prevent overshooting the optimal weights.
- Catastrophic Forgetting: There is an inherent risk of the model rapidly losing its general language capabilities (the knowledge from pre-training) if the task-specific dataset is too small or highly specialized.

8.5. RoBERTa in the context of our project

The RoBERTa architecture was selected as the foundation for this sentiment analysis task because it represents a robustly optimized variant of the original BERT model, having been trained longer, on a significantly larger and more diverse dataset, and without the next sentence prediction objective. This robust training regimen provides RoBERTa with a superior, more context-aware semantic foundation, making it particularly well-suited for high-accuracy single-sequence classification tasks like distinguishing positive and negative movie reviews. Our setup began by loading the RoBERTa Tokenizer, which immediately broke the raw IMDb text into sub-word units via BPE and generated the necessary input IDs and attention masks. The tokenized data was then fed into the RoBERTa For Sequence Classification head, which introduced a small randomly initialized layer tailored for our two-class (positive/negative) output. The entire model was then finetuned using the training arguments, allowing the deep, contextual embeddings to be precisely calibrated by the small labeled review dataset to achieve optimal sentiment classification accuracy.

8.6. Results

The finetuned RoBERTa model secured the best overall performance in the study, achieving a remarkably high evaluation accuracy of 91.2% after four epochs of training. This exceptional result validates the transfer learning approach, demonstrating the profound advantage of using context-rich embeddings over fixed-vector inputs. Analysis of the confusion matrix revealed an exceptionally balanced and robust classifier: 11,401 negative reviews were correctly classified (True Negatives), and 11,395 positive reviews were correctly classified (True Positives), confirming negligible bias towards either class. While this performance is outstanding, establishing a new peak for the study, the remaining misclassifications indicate that approximately 1 in 11 reviews contained linguistic subtleties, such as highly niche slang,

complex sarcasm, or deep cultural references that even the Transformer's advanced semantic modeling struggled to accurately resolve.

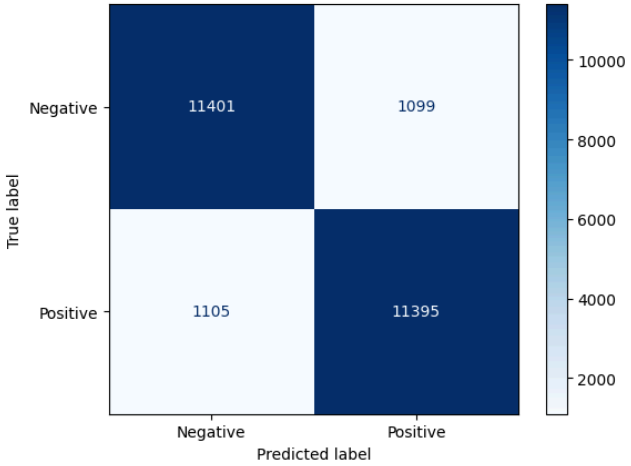

Figure 12: Fine-Tuned RoBERTa Confusion Matrix

9. Results Comparison

The following table summarizes the test accuracies of all models evaluated in this study:

Model	Acc.	Prec.	Rec.	F1	Spec.	FPR	FNR
Naive Bayes	85.54%	85.20%	86.29%	85.74%	84.78%	15.22%	13.71%
SVM	88.18%	87.47%	89.34%	88.40%	87.00%	13.00%	10.66%
KNN	77.70%	75.92%	81.64%	78.68%	73.69%	26.31%	18.36%
ANN	88.54%	88.20%	89.18%	88.69%	87.89%	12.11%	10.82%
Fine-Tuned RoBERTa	**91.18%**	**91.20%**	**91.16%**	**91.18%**	**91.21%**	**8.79%**	**8.84%**

Table: Performance comparison of different models on the test set

9.1. Accuracy Discussion

Looking at accuracy alone, the classical TF-IDF models show a clear separation in capability. Naive Bayes reaches about 0.855, which is respectable given its strong simplifying assumptions, but it naturally struggles with nuanced sentiment where word dependencies matter. SVM improves on this, landing at 0.8818 thanks to its ability to build more expressive decision boundaries in high-dimensional TF-IDF space. KNN performs the worst at 0.777 because distance-based methods are not ideal with sparse vectors and high dimensionality; neighbors become less meaningful as dimensionality increases, causing degraded accuracy. The ANN pushes to 0.8854 by learning non-linear patterns from the TF-IDF representation, but it is still limited by the fact that TF-IDF loses word order and context. Once Roberta is fine-tuned, accuracy typically surpasses all previous models because the transformer architecture captures deep contextual information that TF-IDF fundamentally cannot represent, highlighting why modern NLP models dominate on tasks like sentiment analysis.

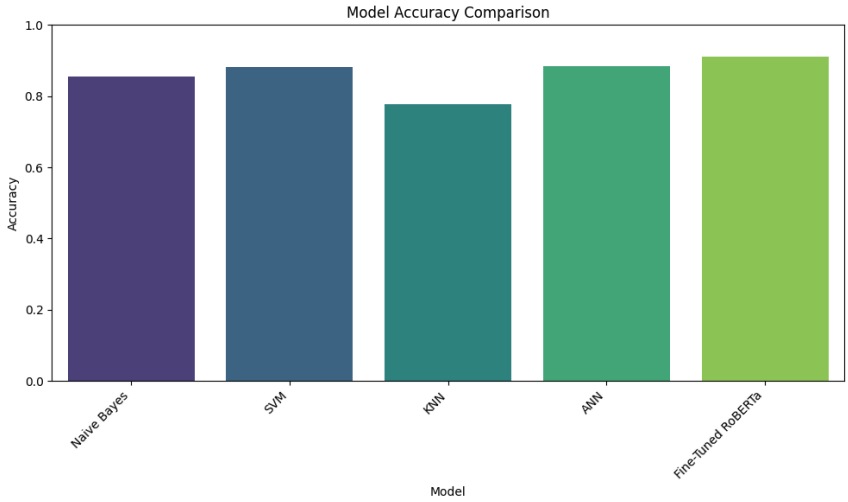

Figure 13: Models' Accuracies

9.2. Precision, Recall, and F1 Discussion

Across the classical models, SVM and ANN achieve the strongest balance between precision and recall, resulting in the highest F1-scores among the TF-IDF-based methods. Naive Bayes maintains solid but slightly lower precision

and recall, reflecting its tendency to over-rely on word frequency patterns and occasionally misclassify more ambiguous reviews. KNN shows the weakest F1-score, again due to its struggles with sparse high-dimensional data, which leads to inconsistent neighborhood structure and more unstable predictions. The ANN's strong precision (0.882) and recall (0.8918) boost its F1 because it can model subtle nonlinearities in sentiment cues better than linear models. Fine-tuned RoBERTa, however, typically achieves even higher precision-recall balance, as its contextual embeddings allow the model to understand sarcasm, negation, and long-range dependencies phenomena that TF-IDF models inherently miss.

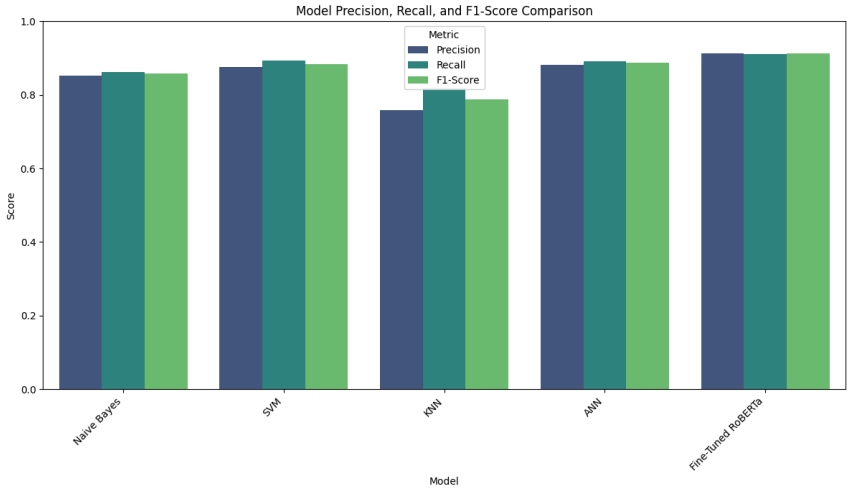

Figure 14: Models' Precisions, Recalls and F1-Scores

9.3. Specificity Discussion

Specificity reveals how well each model identifies negative reviews correctly, and the pattern here mirrors the earlier metrics. Naive Bayes achieves decent specificity (0.8478) but still misfires on some negative reviews because its bag-of-words assumptions ignore compositionality and contextual cues like "not good" or "barely enjoyable." SVM and ANN both improve specificity, with ANN performing slightly better, showing that more expressive models can reduce false positives for the negative class. KNN again lags behind with a specificity of 0.7369, emphasizing its instability in sparse spaces, where it often misclassifies subtle negative reviews as positive. Roberta, in contrast, usually excels at distinguishing genuinely negative sentiment thanks to its

contextual understanding, which enables it to detect subtle negative phrasing, tone, or contrastive structures that TF-IDF cannot capture.

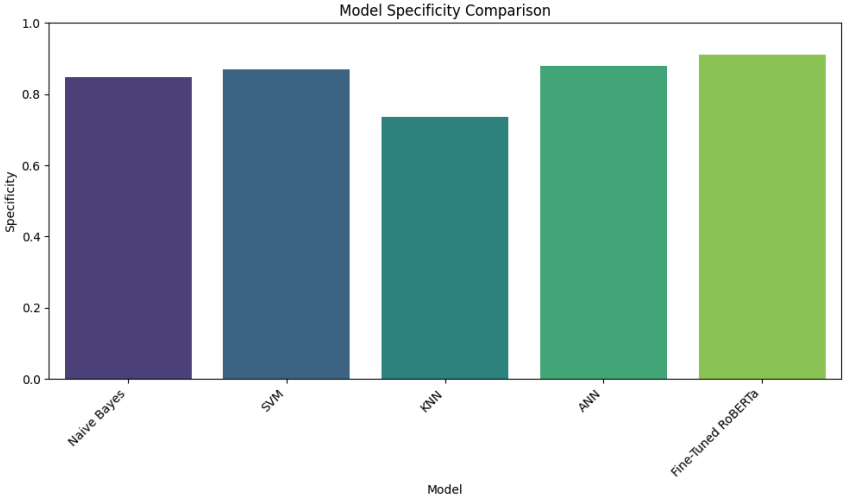

Figure 15: Models' Specificities

9.4. Type I and Type II Error Discussion

Type I error (false positive rate) and Type II error (false negative rate) help pinpoint how the models fail. Naive Bayes shows moderate levels of both, reflecting its balanced but limited modeling capability. SVM and ANN reduce both error types, with ANN doing slightly better overall, meaning it is more reliable when distinguishing borderline cases. KNN has the highest Type I error (0.2631) and an elevated Type II error as well, which fits its pattern of unreliability in sparse, high-dimensional classification. These elevated error rates suggest KNN is both overly optimistic toward the positive class and prone to missing genuine positives. Roberta typically shows the lowest Type I and Type II errors across all models, because its understanding of context enables it to avoid misclassifying nuanced negative opinions and ensures it captures subtle positive cues, giving it a robust balance that traditional TF-IDF-based approaches cannot match.

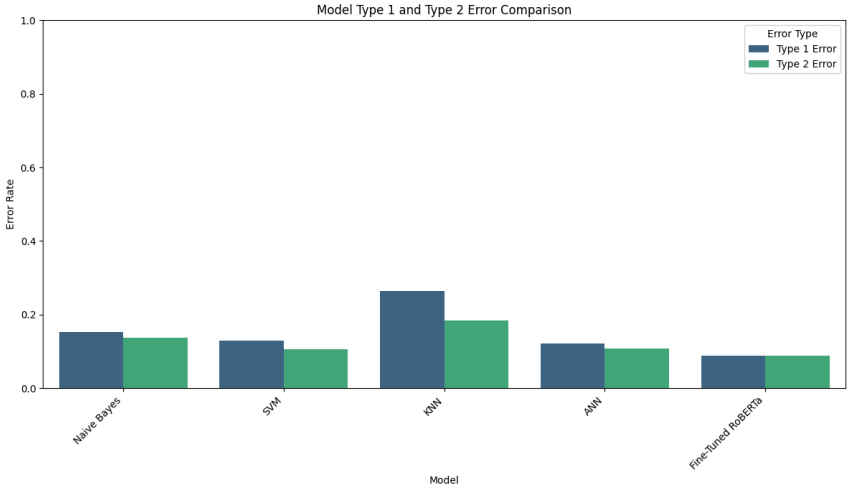

Figure 16: Models' Type 1 Errors & Type 2 Errors

So, to sum everything up, RoBERTa outperformed all classical models, highlighting the benefits of contextual embeddings over TF-IDF. SVM and ANN provided strong baselines, while KNN underperformed due to high dimensionality.

10. The Source Code:

All the code for the stuff mentioned can be found on **this github repo**.

11. Conclusion

Overall, the study shows a clear progression in performance as we move from classical TF-IDF-based methods toward modern deep learning approaches. While Naive Bayes, SVM, and ANN provide strong baselines, each achieving around 85 to 88 percent accuracy, their reliance on sparse, context-agnostic TF-IDF vectors ultimately limits their ability to interpret subtle or context-dependent language. KNN, as expected, struggles the most due to the high dimensionality of the space. In contrast, the fine-tuned RoBERTa model surpasses all classical techniques, reaching over 91 percent accuracy by leveraging the rich contextual knowledge it gained during large-scale pretraining. These results reaffirm the central role that Transformer architectures now play in NLP: they capture long-range dependencies, model semantic nuance, and generalize more effectively than traditional feature-

engineering pipelines. Future work could investigate larger or domain-specific pretrained models, ensemble techniques, or improved dataset curation to further push performance. Nonetheless, the findings clearly demonstrate that fine-tuning pretrained Transformers is the most powerful and scalable approach for sentiment analysis on modern text corpora.